

SAGE Setup Guide

This guide is for the project in the current **SAGE** folder.

All commands below assume you are inside:

```
cd C:\Users\athar\OneDrive\Desktop\B.E\SAGE
```

1. Prerequisites

- Python 3.10 or newer
- **pip**
- PowerShell or a terminal of your choice
- Optional: NVIDIA GPU/CUDA for faster PPO training

The codebase is written for Python 3.10+ and already includes trained model artifacts under **models/**, so you can either:

- use the bundled models for a quick run, or
- retrain the full pipeline from scratch

2. Create a virtual environment

Windows PowerShell

```
py -3 -m venv .venv  
.venv\Scripts\Activate.ps1  
python -m pip install --upgrade pip  
pip install -r requirements.txt
```

If **py** is not available, use:

```
python -m venv .venv
```

If PowerShell blocks activation scripts, run:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

and then activate the environment again.

3. Verify the installation

Run a quick smoke test:

```
python main.py --mode demo
```

Run the unit tests:

```
python -m pytest src\tests\test_sage.py -q
```

If `pytest` is missing, reinstall the project requirements:

```
pip install -r requirements.txt
```

4. Common ways to run the project

Option A: Use the bundled trained models

This is the fastest way to see SAGE working without retraining.

Run the explanation demo:

```
python main.py --mode eval
```

Run scheduler comparison experiments:

```
python main.py --mode experiment
```

Option B: Train everything from scratch

This runs the full pipeline:

1. Collect baseline data
2. Train the Digital Twin predictor
3. Train the PPO scheduler
4. Build the full SAGE agent
5. Run experiments
6. Print explanation examples

```
python main.py --mode full --timesteps 50000
```

For a shorter training run during development:

```
python main.py --mode full --timesteps 5000
```

Option C: Train only

```
python main.py --mode train --timesteps 50000
```

Option D: Standalone PPO training

```
python -m src.agents.train_ppo --timesteps 50000 --save-path models/ppo_sage
```

5. Real-time dashboard

The project also includes a Flask dashboard with live simulation updates.

Start it with:

```
python simulation_server.py
```

Then open:

- <http://localhost:5000>

The dashboard loads:

- `models/dt_predictor.joblib`
- `models/ppo_sage/ppo_sage_final.zip`

If those files are missing or outdated, run a training command first.

6. Important output folders

- `models/` stores the Digital Twin and PPO model files
- `logs/experiments/` stores experiment CSVs and generated plots
- `logs/tb_sage/` stores TensorBoard logs

To inspect TensorBoard logs:

```
tensorboard --logdir logs/tb_sage
```

7. Useful commands

```
python main.py --mode demo
python main.py --mode eval
python main.py --mode experiment
python main.py --mode train --timesteps 50000
python main.py --mode full --timesteps 50000
python -m pytest src\tests\test_sage.py -q
python simulation_server.py
```

8. Troubleshooting

No module named pytest

Install the project dependencies again:

```
pip install -r requirements.txt
```

Model load errors in `eval`, `experiment`, or dashboard mode

Regenerate the saved models:

```
python main.py --mode train --timesteps 50000
```

SHAP import problems

`shap` is listed in `requirements.txt`. If it fails to import, reinstall dependencies. The explainer can still fall back to model feature importance when the Digital Twin is fitted.

PPO training is slow

Reduce timesteps for local testing:

```
python main.py --mode full --timesteps 5000
```

9. Recommended first run

If you just want to confirm the project works in this folder, use this order:

```
python main.py --mode demo
python main.py --mode eval
python simulation_server.py
```

That gives you:

- a basic simulator check
- a run using the saved SAGE models
- the live dashboard view